ЛАБОРАТОРНАЯ РАБОТА №7: КОЛЛЕКТИВНЫЕ ОПЕРАЦИИ MPI

Выполнил: Заика Тимур, группа 11-404

1. Постановка задачи и реализация

Целью работы является освоение базовых коллективных операций MPI на примере задачи вычисления суммы элементов массива. Программа распределяет массив между процессами, каждый процесс считает локальную сумму, после чего общий результат собирается тремя различными способами: через `MPI_Gather` (с ручным суммированием на корне), через `MPI_Reduce` и через `MPI_Allreduce`.

В отличие от базового варианта, программа поддерживает случай, когда размер массива N не делится на число процессов P без остатка. Для этого используется коллективная операция `MPI_Scatterv`, позволяющая передавать процессам блоки разного размера (разница в 1 элемент). Время измеряется с помощью `MPI_Wtime`.

Операция MPI	Назначение в программе	Кто получает результат
MPI_Bcast	Передача параметров (N, runs, reps) от корня всем процессам	Все процессы
MPI_Scatter / Scatterv	Распределение элементов массива между процессами	Все процессы (каждый свою часть)
MPI_Gather	Сбор локальных сумм в массив на процессе 0	Только процесс 0
MPI_Reduce	Свертка локальных сумм с операцией MPI_SUM	Только процесс 0
MPI_Allreduce	Свертка локальных сумм с операцией MPI_SUM	Все процессы

2. Компиляция и запуск

Компилятор: GCC 11.3, реализация MPI: OpenMPI 4.1. Тестирование проводилось на процессоре Apple Silicon M-серии.

- `mpicc mpi_sum_custom.c -o mpi_sum_custom`
- `mpiexec -n 4 ./mpi_sum_custom 1000000 # N=10^6, P=4`
- `mpiexec -n 4 ./mpi_sum_custom 1000003 5 10000 # N не делится на 4, 5 запусков, 10000 повторений`

Таблица 4. Сравнение Gather и Reduce (N = 1 000 000, 10000 повторений)

Число процессов P	Способ агрегирования	Среднее время операции, с	Результат корректен?
4	Gather + сумма на процессе 0	0.0000046	Да
4	Reduce	0.0000032	Да
8	Gather + сумма на процессе 0	0.0000071	Да
8	Reduce	0.0000045	Да

4. Вывод

В ходе выполнения работы были успешно применены коллективные операции MPI. Как показала Таблица 1, все три способа агрегации (`Gather` , `Reduce` , `Allreduce`) дают идентичный и математически корректный результат, совпадающий с формулой *N(N+1)/2*. Также было подтверждено, что использование `MPI_Scatterv` корректно обрабатывает случай, когда размер массива не кратен числу процессов.

Анализ производительности (Таблица 3) показывает, что ускорение растет вплоть до 4 процессов, достигая значения 3.24 при эффективности 81%. Дальнейшее увеличение числа процессов до 8 приводит к падению эффективности до 37% и легкому замедлению. Это объясняется тем, что время, затрачиваемое на коллективные пересылки и синхронизацию, начинает превышать время полезных вычислений (локального суммирования).

Сравнение механизмов агрегации (Таблица 4) показывает, что `MPI_Reduce` работает стабильно быстрее, чем `MPI_Gather` с ручным суммированием. Это связано с тем, что `Reduce` использует внутренние оптимизированные алгоритмы свертки (например, древовидные), а при использовании `Gather` корневой процесс вынужден после сбора данных выполнять дополнительный последовательный цикл.

Таким образом, выбор операции зависит от задачи: **Gather** незаменим, когда нужны все промежуточные результаты; **Reduce** оптимален для получения одного итогового значения на корневом процессе; а **Allreduce** удобен, когда итог вычислений требуется всем процессам одновременно.

3. Результаты

Таблица 1. Проверка корректности вычисления суммы

P	N	Ожидаемая сумма	Gather	Reduce	Allreduce	Корректно?
2	100	5050	5050	5050	5050	Да
4	1 000 000	500000500000	500000500000	500000500000	500000500000	Да
4	1 000 003	500003500006	500003500006	500003500006	500003500006	Да
8	10 000 000	50000005000000	50000005000000	50000005000000	50000005000000	Да

Таблица 2. Влияние размера массива на время (P = 4)

N	Время выполнения, с	Результат корректен?
10 000	0.0004	Да
100 000	0.0012	Да
1 000 000	0.0095	Да
10 000 000	0.0880	Да

Таблица 3. Влияние числа процессов (N = 10 000 000)

Число процессов P	Время T_P, с	Ускорение S_P	Эффективность E_P
1	0.2850	1.00	1.00
2	0.1500	1.90	0.95
4	0.0880	3.24	0.81
8	0.0960	2.97	0.37